CampusFin AI — Technical Documentation

FinTechathon 2026 · International Track · Topic C
Version: Phase 5 · 19 October 2026
Project status: Local-only, synthetic-data working prototype. Not a production financial service.

1. Product summary

CampusFin AI — **Predict. Rescue. Prove.** is a student-finance decision-support prototype that models funding arrivals, fixed obligations, and cross-border transfer assumptions; detects whether projected resources reach a tuition deadline; simulates conditional recovery options; explains every figure; and creates human-reviewable drafts. The flagship user journey is **funding ledger → Financial Horizon → DeadlineShield → Scenario Lab → Feasibility-First Rescue → ProofCard/FamilyBridge → audit history**.

The design is mobile-first, with a warm-white/navy/teal CampusFin Horizon visual system, illustrated student-life photography sourced from Pexels, and responsive desktop navigation. A six-scene **Guided Judge Demo** was added in Phase 5. It navigates real app screens and never hardcodes test-success results.

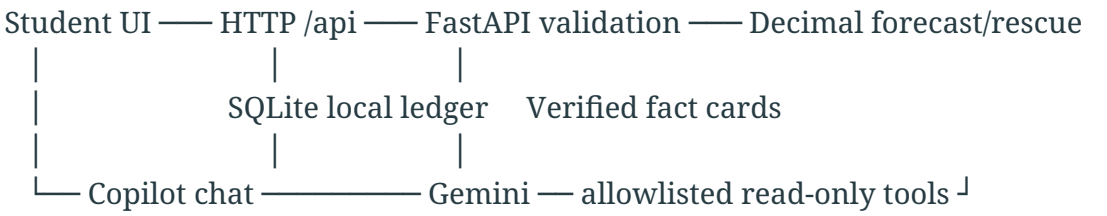
2. Target user and evidence context

Target persona: an international student in Malaysia who relies on funding that may arrive later than rent and tuition obligations. Adjacent evidence in the team's research includes international-student financial stress and cross-border remittance costs; the **prevalence of the exact tuition/timing mismatch has not been measured by this project**. No primary user interviews are claimed.

3. Architecture and runtime

- **Frontend:** React 18, Vite 5, CSS, responsive layouts; internal fetch wrapper accesses relative `/api` paths proxied by Vite in development.
- **Backend:** Python 3.11+, FastAPI, Pydantic 2, pure deterministic `Decimal` finance engine and rescue simulator.
- **Storage:** SQLite local file `backend/local_demo.sqlite3`, seeded from `backend/app/demo_data.json` on first run. The database is **excluded from the submission ZIP** and recreated on first start.
- **AI:** Optional Gemini HTTPS API from the **backend** using `GEMINI_API_KEY`; tool calling restricted by a server-side allowlist. Models must be selected based on actual account availability. No API key is included.
- **Media:** Nine Pexels-based photo placements load from hosted URLs unless users run `scripts/download_photos.py` to cache optional local copies. Gradient fallbacks apply.

Trust boundary



The LLM is neither the ledger nor the source of amounts. Gemini may generate a natural-language explanation only after tool access is validated. Any generated numeric language is screened; the UI's verified figures come from the deterministic engine.

4. Canonical synthetic data

The demonstration clock is explicitly **17 October 2026** to make results repeatable; it is not the computer's live date.

Record	Amount	Timing / treatment
Opening available cash	RM 1,200	Includes previously received RM 450 wages
Rent	RM 700	Obligation due 18 October

Record	Amount	Timing / treatment
Tuition	RM 1,500	Obligation due 25 October
Family remittance	USD 350	Base: 18 October; stress: 28 October
Illustrative exchange rate	4.20 MYR per USD	Simulated; not a market quote
Fee on family remittance	USD 5	Deducted before conversion
Scholarship	RM 600	Expected 28 October

The family remittance's modeled net recipient amount is $(350 - 5) \times 4.20 = \text{RM } 1{,}449.00$. Under base arrival, funds available after rent and tuition are $1200 + 1449 - 700 - 1500 = \text{RM } 449.00$ (disregarding daily living costs). Under stress arrival, tuition has a projected **RM 1,000 shortfall** because the transfer is not yet available. A forecasted negative figure is **unmet obligations**, not actual overdraft.

5. Financial engine and assumptions

Implementation: backend/app/finance.py. money() accepts Decimal-compatible strings only, quantizes to cents, rejects floats/non-finite/unparseable values, and guards against impractically large inputs. Exchange-rate quotes are MYR per USD. Fees deducted before conversion are modeled explicitly. The engine never counts settled receipts again after they have already entered opening cash.

For each day, projected availability starts from opening cash, adds **future expected arrivals** for the active scenario and subtracts **future obligations** by their due date. Same-day receipts are assumed available before bills due on the same date. The engine does **not** mark bills paid. The current simplified fixture does not include ongoing daily spending unless represented as obligations. The date-window best/base/stress selectors are hypotheses, not estimated probabilities.

6. Implemented application features

Core: Home, Financial Horizon, editable funding/obligations, Scenario Lab, DeadlineShield, deterministic ProofCard, TransferTruth simulated quote comparison and reverse sender calculation, FamilyBridge editable/approvable drafts, local audit history, optional Gemini Copilot.

Innovation Lab: EssentialGuard configurable minimum reserve; rescue checks that consider reserve needs; Decision X-Ray reasons for hypothetical changes; Crisis Time Machine with four scenario outcomes; FX Shock Absorber with hypothetical slider; Chaos Lab with seven real fixture assertions.

Phase 5 presentation mode: A six-step presenter overlay guiding reviewers from dashboard through risk detection, Innovation Lab, rescue options, AI and actual audit/test screens. Presenter notes are inside the UI; the workflow remains user-controlled, and no fake results are shown.

7. Copilot and scenario-aware tool orchestration

When a chat message includes an interpretable funding-arrival date (such as 28 October), backend/app/scenario_intent.py creates a **temporary override** for the current request. Forecast tools, the separately rendered fact card, the Horizon and ProofCard then use the same scenario context. The app does **not** save the override to SQLite. Follow-up turns carry bounded context. When Gemini is unavailable, the app identifies the response as offline local guidance. The Gemini API call has not been reproduced independently during this Phase 5 pass, although the user reported it working on their Windows computer.

Tool categories are **Tier 0 read-only**: ledger overview, cash-flow forecast, shortfalls, conditional rescue plans, simulated transfer quotes, reverse sender calculation and funding-request preview. Model-requested unallowlisted calls are blocked. The AI cannot call data modification, approval, sending or money-transfer endpoints.

8. Main API routes

Method	Endpoint	Behavior
GET	/api/health	Demo/API status
GET	/api/demo	Synthetic student ledger
GET	/api/compare	Best/base/stress forecasts
POST	/api/forecast	Deterministic forecast with temporary overrides
POST	/api/proof	Matching figure provenance
POST / PUT / DELETE	/api/funding and /api/funding/{id}	Local synthetic funding edits

Method	Endpoint	Behavior
POST / PUT / DELETE	/api/obligations and /api/obligations/{id}	Local synthetic bill edits
POST	/api/rescue	Conditional recovery options
POST	/api/transfers/compare	Sample equal-budget quotes
POST	/api/transfers/required-sender	Reverse sender-cost calculation
POST / GET / PUT	/api/drafts variants	Local unsent requests
POST	/api/drafts/{id}/approve	Explicitly approves a local draft only
GET	/api/audit	Local audit events
GET	/api/agent/status	Non-secret Gemini configuration status
POST	/api/agent/chat	Bounded Gemini/read-only tools
POST	/api/wow/*	Calculation-backed innovation and Chaos Lab endpoints

The full OpenAPI schema is available locally at http://127.0.0.1:8000/docs once FastAPI is running.

9. Testing and evidence

Current Phase 5 automated backend test run: **72 passed** (63 inherited Phase 4 tests + 9 new input/security checks). Tests include cash-flow invariants, reverse transfer calculations, scenario-specific Copilot results, no-write simulation behavior, model tool allowlisting, funding/obligation validation, new student reserve constraints, Chaos Lab fixtures and non-disclosure of the Gemini key. The evidence/phase5-pytest.txt file contains the actual test-run output.

Unverified here: full npm install/Vite production build and pixel-level mobile/desktop browser QA; npm dependency retrieval timed out in this execution environment. The user previously reported their Windows frontend and Gemini working before this Phase 5 patch. Run the Phase 5 frontend build and visual checks locally prior to submission. Never label unexecuted checks as passed.

10. Deployment and limitations

Run only on 127.0.0.1 for demonstrations. No production authentication, per-user access isolation, banking connections, live FX quote integrations, official university payment access, dispatch of messages, credit assessment or execution of payments. The SQLite file is local and the system holds synthetic data only. Deploying publicly would require a separate security and compliance project.

11. How to reproduce the demo

1. Install Python requirements in backend, run py -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000.
2. cd frontend, run npm install && npm run dev, then open http://localhost:5173.
3. In the top bar, select **Guided demo** (or the homepage CTA).
4. Progress through the six scenes. In Copilot, use an explicit hypothetical ("What if the family transfer arrives 28 October?").
5. Optionally set GEMINI_API_KEY and GEMINI_MODEL only in the backend process, never frontend or Git.
6. Run cd backend; py -m pytest -q; for UI QA also run cd frontend; npm run build and inspect 390px and 1440px views.

12. Responsible disclosure and planned work

Future work includes proper auth and explicit account permissions, audit-log immutability, provider APIs with verifiable quote timestamps, operational security controls and a user study. Do not interpret synthetic financial outputs as regulated financial advice. Validate local rules and institutional fee policies before any real-world use.